

Sigaction vs Signal



Difference b/w sigaction and signal



The difference between `signal` and `sigaction` lies in their functionality and flexibility when handling signals in Unix-like operating systems:

1. `signal` Function:

- A simpler, older interface for setting signal handlers.
- It allows you to specify a function to handle a particular signal, or to ignore or restore the default signal handling.
- Its behavior varies across different systems and implementations, which can lead to portability issues.
- Limited control over signal handling options.
- Example:

c

```
signal(SIGINT, handler_function);
```

2. `sigaction` Function:

- A more modern and robust way of handling signals, offering more control and flexibility.
- It allows you to specify additional options for signal handling, such as blocking certain signals while the handler is executing, or restoring the signal mask after the handler completes.
- Provides reliable and consistent behavior across different platforms.
- It can manage flags like `SA_RESTART`, which automatically restarts certain system calls if interrupted by a signal.
- Example:

c

```
struct sigaction sa;  
sa.sa_handler = handler_function;  
sigemptyset(&sa.sa_mask);  
sa.sa_flags = SA_RESTART; // Restart interrupted system calls  
sigaction(SIGINT, &sa, NULL);
```

In summary, `sigaction` is the preferred and more reliable method due to its flexibility and portability, while `signal` is a simpler, older alternative with less control.

ChatGPT can make mistakes. Check important info.